

JOB SHEET 12

Nim: 25410720081

Nama: M Zainur Roziqin

kode di class node23.java:

```
package Pertemuan12.Jobsheet12;

public class Node23 {
    Mahasiswa23 data;
    Node23 prev;
    Node23 next;

    public Node23(Node23 prev, Mahasiswa23 data, Node23 next) {
        this.prev = prev;
        this.data = data;
        this.next = next;
    }
}
```

kode di class Mahasiswa23.java:

```
package Pertemuan12.Jobsheet12;

public class Mahasiswa23 {
    String nim;
    String nama;
    String kelas;
    double ipk;

    public Mahasiswa23(String nim, String nama, String kelas, double ipk) {
        this.nim = nim;
        this.nama = nama;
        this.kelas = kelas;
        this.ipk = ipk;
    }

    void tampil() {
        System.out.println("NIM : " + nim);
        System.out.println("Nama : " + nama);
        System.out.println("Kelas : " + kelas);
        System.out.println("IPK : " + ipk);
    }
}
```

```
}  
}
```

Kode di class DoubleLinkedList23.java:

```
package Pertemuan12.Jobsheet12;  
  
public class DoubleLinkedList23 {  
    Node23 head;  
    Node23 tail;  
  
    public DoubleLinkedList23() {  
        head = null;  
        tail = null;  
    }  
  
    boolean isEmpty() {  
        return head == null;  
    }  
  
    int size() {  
        int count = 0;  
        Node23 current = head;  
        while (current != null) {  
            count++;  
            current = current.next;  
        }  
        return count;  
    }  
  
    void addFirst(Mahasiswa23 data) {  
        Node23 newNode = new Node23(null, data, head);  
        if (isEmpty()) {  
            head = tail = newNode;  
        } else {  
            head.prev = newNode;  
            head = newNode;  
        }  
    }  
  
    void addLast(Mahasiswa23 data) {  
        Node23 newNode = new Node23(tail, data, null);
```

```

        if (isEmpty()) {
            head = tail = newNode;
        } else {
            tail.next = newNode;
            tail = newNode;
        }
    }

    void add(Mahasiswa23 data, int index) {
        if (index < 0) {
            System.out.println("Index tidak valid.");
            return;
        }
        if (index == 0) {
            addFirst(data);
            return;
        }

        int currentSize = size();
        if (index > currentSize) {
            System.out.println("Index melebihi panjang linked list.");
            return;
        }
        if (index == currentSize) {
            addLast(data);
            return;
        }

        Node23 current = head;
        int i = 0;
        while (current != null && i < index) {
            current = current.next;
            i++;
        }

        if (current == null) {
            System.out.println("Index melebihi panjang linked list.");
            return;
        }

        Node23 newNode = new Node23(current.prev, data, current);
        current.prev.next = newNode;
        current.prev = newNode;
    }

```

```

    }

    Node23 search(String keyNim) {
        Node23 current = head;
        while (current != null) {
            if (current.data.nim.equalsIgnoreCase(keyNim)) {
                return current;
            }
            current = current.next;
        }
        return null;
    }

    void insertAfter(String keyNim, Mahasiswa23 data) {
        Node23 current = search(keyNim);
        if (current == null) {
            System.out.println("Data dengan NIM " + keyNim + " tidak
ditemukan.");
            return;
        }

        if (current == tail) {
            addLast(data);
            return;
        }

        Node23 newNode = new Node23(current, data, current.next);
        current.next.prev = newNode;
        current.next = newNode;
    }

    void print() {
        if (isEmpty()) {
            System.out.println("Linked List masih kosong");
            return;
        }

        System.out.println("Isi Double Linked List:");
        Node23 current = head;
        int no = 1;
        while (current != null) {
            System.out.println("Data ke-" + no);
            current.data.tampil();
        }
    }
}

```

```

        System.out.println();
        current = current.next;
        no++;
    }
}

void printReverse() {
    if (isEmpty()) {
        System.out.println("Linked List masih kosong");
        return;
    }

    System.out.println("Isi Double Linked List (reverse):");
    Node23 current = tail;
    int no = size();
    while (current != null) {
        System.out.println("Data ke-" + no);
        current.data.tampil();
        System.out.println();
        current = current.prev;
        no--;
    }
}

Mahasiswa23 removeFirst() {
    if (isEmpty()) {
        System.out.println("Linked List masih kosong");
        return null;
    }

    Mahasiswa23 removedData = head.data;
    if (head == tail) {
        head = tail = null;
    } else {
        head = head.next;
        head.prev = null;
    }

    System.out.println("Data berhasil dihapus dari awal:");
    removedData.tampil();
    return removedData;
}

```

```

Mahasiswa23 removeLast() {
    if (isEmpty()) {
        System.out.println("Linked List masih kosong");
        return null;
    }

    Mahasiswa23 removedData = tail.data;
    if (head == tail) {
        head = tail = null;
    } else {
        tail = tail.prev;
        tail.next = null;
    }

    System.out.println("Data berhasil dihapus dari akhir:");
    removedData.tampil();
    return removedData;
}
}

```

kode di class DoubleLinkedListMain23.java:

```

package Pertemuan12.Jobsheet12;

import java.util.Scanner;

public class DoubleLinkedListMain23 {
    static String readLineNonEmpty(Scanner sc, String prompt) {
        String value;
        do {
            System.out.print(prompt);
            value = sc.nextLine().trim();
        } while (value.isEmpty());
        return value;
    }

    static int readInt(Scanner sc, String prompt) {
        System.out.print(prompt);
        while (!sc.hasNextInt()) {
            sc.next();
            System.out.print(prompt);
        }
        int value = sc.nextInt();
    }
}

```

```

        sc.nextLine();
        return value;
    }

    static double readDoubleFlexible(Scanner sc, String prompt) {
        System.out.print(prompt);
        String token = sc.nextLine().trim().replace(',', '.', '.');
        while (token.isEmpty()) {
            System.out.print(prompt);
            token = sc.nextLine().trim().replace(',', '.', '.');
        }
        return Double.parseDouble(token);
    }

    static Mahasiswa23 inputMahasiswa(Scanner sc) {
        String nim = readLineNonEmpty(sc, "NIM : ");
        String nama = readLineNonEmpty(sc, "Nama : ");
        String kelas = readLineNonEmpty(sc, "Kelas : ");
        double ipk = readDoubleFlexible(sc, "IPK : ");
        return new Mahasiswa23(nim, nama, kelas, ipk);
    }

    static void menu() {
        System.out.println("\n=== Menu Double Linked List ===");
        System.out.println("1. Tambah data di awal");
        System.out.println("2. Tambah data di akhir");
        System.out.println("3. Tambah data pada index tertentu");
        System.out.println("4. Sisipkan data setelah NIM tertentu");
        System.out.println("5. Cari data berdasarkan NIM");
        System.out.println("6. Tampilkan seluruh data");
        System.out.println("7. Tampilkan data secara terbalik");
        System.out.println("8. Hapus data paling awal");
        System.out.println("9. Hapus data paling akhir");
        System.out.println("0. Keluar");
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        DoubleLinkedList23 list = new DoubleLinkedList23();
        int pilihan;

        do {
            menu();

```

```

        pilihan = readInt(sc, "Pilih menu: ");

        switch (pilihan) {
            case 1:
                list.addFirst(inputMahasiswa(sc));
                break;
            case 2:
                list.addLast(inputMahasiswa(sc));
                break;
            case 3: {
                Mahasiswa23 data = inputMahasiswa(sc);
                int index = readInt(sc, "Index: ");
                list.add(data, index);
                break;
            }
            case 4: {
                String keyNim = readLineNonEmpty(sc, "Sisip setelah
NIM: ");

                Mahasiswa23 data = inputMahasiswa(sc);
                list.insertAfter(keyNim, data);
                break;
            }
            case 5: {
                String keyNim = readLineNonEmpty(sc, "Cari NIM: ");
                Node23 found = list.search(keyNim);
                if (found == null) {
                    System.out.println("Data tidak ditemukan.");
                } else {
                    System.out.println("Data ditemukan:");
                    found.data.tampil();
                }
                break;
            }
            case 6:
                list.print();
                break;
            case 7:
                list.printReverse();
                break;
            case 8:
                list.removeFirst();
                break;
            case 9:

```



```

        list.removeLast();
        break;
    case 0:
        System.out.println("Keluar.");
        break;
    default:
        System.out.println("Menu tidak tersedia.");
    }
} while (pilihan != 0);

sc.close();
}
}

```

verifikasi hasil:

TambahDataDiakhir():

```

2. Tambah data di akhir
3. Tambah data pada index tertentu
4. Sisipkan data setelah NIM tertentu
5. Cari data berdasarkan NIM
6. Tampilkan seluruh data
7. Tampilkan data secara terbalik
8. Hapus data paling awal
9. Hapus data paling akhir
0. Keluar
Pilih menu: 2
NIM   : 105
Nama  : erij
Kelas : 1g
IPK   : 3

```

Sisipkan setelah NIM:

```

3. Tambah data pada index tertentu
4. Sisipkan data setelah NIM tertentu
5. Cari data berdasarkan NIM
6. Tampilkan seluruh data
7. Tampilkan data secara terbalik
8. Hapus data paling awal
9. Hapus data paling akhir
0. Keluar
Pilih menu: 4
Sisip setelah NIM: 101
NIM   : 1001
Nama  : rozi
Kelas : 1g
IPK   : 4

```

Tampilkan data:

```

Pilih menu: 0
Isi Double Linked List
Data ke-1
NIM   : 103
Nama  : anjay
Kelas : 1g
IPK   : 3.0

Data ke-2
NIM   : 102
Nama  : obi
Kelas : 1g
IPK   : 4.0

Data ke-3
NIM   : 101
Nama  : rozi
Kelas : 1g
IPK   : 4.0

Data ke-4
NIM   : 1001
Nama  : rozi
Kelas : 1g
IPK   : 4.0

```

```

NIM   : 102
Nama  : obi
Kelas : 1g
IPK   : 4.0

Data ke-3
NIM   : 101
Nama  : rozi
Kelas : 1g
IPK   : 4.0

Data ke-4
NIM   : 1001
Nama  : rozi
Kelas : 1g
IPK   : 4.0

Data ke-5
NIM   : 105
Nama  : erij
Kelas : 1g
IPK   : 3.0

```

hapus data awal:

```

4. Sisipkan data setelah NIM tertentu
5. Cari data berdasarkan NIM
3. Tambah data pada index tertentu
4. Sisipkan data setelah NIM tertentu
5. Cari data berdasarkan NIM
6. Tampilkan seluruh data
7. Tampilkan data secara terbalik
8. Hapus data paling awal
9. Hapus data paling akhir
0. Keluar
Pilih menu: 8
Data berhasil dihapus dari awal:
NIM   : 103
Nama  : anjay
Kelas : 1g
IPK   : 3.0

```

hapus data akhir:

```

9. Hapus data paling akhir
0. Keluar
Pilih menu: 9
Data berhasil dihapus dari akhir:
NIM   : 105
Nama  : erij
Kelas : 1g
IPK   : 3.0

```

print tampil:

```

Pilih menu: 0
Isi Double Linked List
Data ke-1
NIM   : 102
Nama  : obi
Kelas : 1g
IPK   : 4.0

Data ke-2
NIM   : 101
Nama  : rozi
Kelas : 1g
IPK   : 4.0

Data ke-3
NIM   : 1001
Nama  : rozi
Kelas : 1g
IPK   : 4.0

```

12.2.3 jawaban pertanyaan:

1. Perbedaan struktur dan traversal antara Single Linked List dan Double Linked List
 - Single Linked List hanya memiliki pointer next, jadi traversal normal hanya bisa maju dari head ke node berikutnya.

- Double Linked List memiliki pointer next dan prev, jadi traversal bisa maju dari head ke tail maupun mundur dari tail ke head.
- Pada proses sisip/hapus, Double Linked List membutuhkan pengaturan dua arah pointer, sedangkan Single Linked List hanya mengatur hubungan next.

2. Fungsi atribut next dan prev pada class Node

- next menunjuk ke node sesudah node saat ini, sehingga dipakai saat traversal maju.
- prev menunjuk ke node sebelum node saat ini, sehingga dipakai saat traversal mundur.
- Pada manipulasi node, kedua pointer ini menjaga agar hubungan antar node tetap tersambung saat penyisipan maupun penghapusan.

3. Fungsi konstruktor pada class DoubleLinkedList terhadap kondisi awal linked list

- Konstruktor menginisialisasi head dan tail dengan null.
- Ini menandakan linked list masih kosong dan belum memiliki node.
- Kondisi awal tersebut dipakai method isEmpty() untuk membedakan linked list kosong dan tidak kosong.

4. Mengapa head dan tail menunjuk node yang sama saat linked list masih kosong lalu ditambah 1 node

- Karena saat elemen pertama ditambahkan, node itu sekaligus menjadi node pertama dan node terakhir.
- Belum ada node lain di depan maupun di belakangnya.
- Setelah ada penambahan node berikutnya, head dan tail baru bisa menunjuk node yang berbeda.

5. Modifikasi print() agar menampilkan pesan saat linked list kosong

- Sudah diterapkan pada DoubleLinkedList23.java.
- Jika isEmpty() bernilai true, method print() menampilkan Linked List masih kosong.

```
void print() {
    if (isEmpty()) {
        System.out.println("Linked List masih kosong");
        return;
    }

    System.out.println("Isi Double Linked List:");
    Node23 current = head;
    int no = 1;
    while (current != null) {
        System.out.println("Data ke-" + no);
        current.data.tampil();
        System.out.println();
        current = current.next;
        no++;
    }
}
```

6. Modifikasi printReverse() untuk menampilkan data dari tail ke head

- Sudah diterapkan pada DoubleLinkedList23.java.

- Method printReverse() melakukan traversal mundur mulai dari tail, lalu berpindah dengan pointer prev sampai mencapai head.

```
void printReverse() {
    if (isEmpty()) {
        System.out.println("Linked List masih kosong");
        return;
    }

    System.out.println("Isi Double Linked List (reverse):");
    Node23 current = tail;
    int no = size();
    while (current != null) {
        System.out.println("Data ke-" + no);
        current.data.tampil();
        System.out.println();
        current = current.prev;
        no--;
    }
}
```

12.3.3 jawaban pertanyaan:

1. Fungsi head = head.next; dan head.prev = null; pada removeFirst()

- head = head.next; memindahkan penunjuk head ke node kedua, sehingga node pertama lama tidak lagi menjadi awal linked list.

- head.prev = null; memutus hubungan balik dari node baru pertama ke node lama yang sudah dihapus.

- Kombinasi keduanya membuat node pertama terhapus dengan struktur linked list tetap valid.

2. Modifikasi removeFirst() dan removeLast() agar menampilkan data yang dihapus

- removeFirst() menampilkan data yang dihapus dari bagian awal.

- removeLast() menampilkan data yang dihapus dari bagian akhir.

```
Mahasiswa23 removeFirst() {
    if (isEmpty()) {
        System.out.println("Linked List masih kosong");
        return null;
    }

    Mahasiswa23 removedData = head.data;
```

```

        if (head == tail) {
            head = tail = null;
        } else {
            head = head.next;
            head.prev = null;
        }

        System.out.println("Data berhasil dihapus dari awal:");
        removedData.tampil();
        return removedData;
    }

    Mahasiswa23 removeLast() {
        if (isEmpty()) {
            System.out.println("Linked List masih kosong");
            return null;
        }

        Mahasiswa23 removedData = tail.data;
        if (head == tail) {
            head = tail = null;
        } else {
            tail = tail.prev;
            tail.next = null;
        }

        System.out.println("Data berhasil dihapus dari akhir:");
        removedData.tampil();
        return removedData;
    }
}

```

PROGRESS CM 2

kode di class nodepembelian23.java:

```

package CM2;

public class NodePembeli23 {
    Pembeli23 data;
    NodePembeli23 prev;
}

```

```
NodePembeli23 next;

    public NodePembeli23(NodePembeli23 prev, Pembeli23 data,
NodePembeli23 next) {
        this.prev = prev;
        this.data = data;
        this.next = next;
    }
}
```